

AUDIT_RAW_FINDINGS.md — Auditoria Técnica Bruta do Repositório `born-to-run`

Documento de achados brutos. Este arquivo consolida TODOS os achados da auditoria técnica do repositório `samukkaneto/born-to-run`. É um documento de trabalho (raw findings) que servirá de insumo para os documentos finais de auditoria. **Nenhum código-fonte foi alterado**; apenas este arquivo foi criado.

0. Metadados da Auditoria

Item	Valor
Repositório	<code>samukkaneto/born-to-run</code> (público)
Local do clone	<code>/home/ubuntu/github_repos/born-to-run</code>
Branch	<code>main</code>
Profundidade do clone	<code>--depth=50</code>
Data da auditoria	17/07/2026
Total de arquivos-fonte (excl. node_modules/.next/.git)	77
Ambiente	Node v22.14.0, npm 10.9.2
Framework	Next.js 16.2.10 (App Router, Turbopack), React 19.2.4
Backend	Supabase (@supabase/ssr ^0.12.3, @supabase/supabase-js ^2.110.5)
Estilo	Tailwind CSS v4 (@tailwindcss/postcss)

Histórico Git (2 commits)

```
3e4cdf4 First big commit with auth, database, social feed, workouts, profile
d205076 Initial commit from Create Next App
```

O projeto foi desenvolvido essencialmente em **um único commit grande**, sem histórico incremental que permita rastrear decisões.

1. Metodologia

1. Clone do repositório no worktree real (sem `git init`, sem cópia para outro diretório).
 2. Leitura integral de **todos** os arquivos-fonte (documentação, configs, SQL, middleware, actions, páginas, componentes, manifest).
 3. Comparação linha a linha do `HANDOVER.md` (documentação entregue) contra o schema SQL e o código real.
 4. Execução da cadeia de build: `npm install` → `npm audit` → `npm run lint` → `npx tsc --noEmit` → `npm run build`, capturando saídas e erros completos.
 5. Buscas dirigidas (grep) para confirmar exportações, rotas ausentes, papéis (`role`), duplicações e classes CSS indefinidas.
-

2. Lista Completa de Arquivos Lidos

2.1 Documentação / raiz

- `HANDOVER.md`
- `README.md`
- `AGENTS.md`
- `CLAUDE.md`
- `.gitignore`

2.2 Configuração

- `package.json`
- `package-lock.json` (verificado: resolução da `lucide-react`)
- `next.config.ts`
- `tailwind.config.ts`
- `tsconfig.json`
- `postcss.config.mjs`
- `eslint.config.mjs`
- `next-env.d.ts`
- `middleware.ts`

2.3 Tipos e utilitários

- `types/index.ts`
- `lib/utils.ts`

2.4 Integração Supabase / Auth

- `lib/supabase/client.ts`
- `lib/supabase/server.ts`
- `lib/supabase/middleware.ts`
- `app/auth/callback/route.ts`
- `supabase/schema.sql`

2.5 Server Actions (DOIS conjuntos concorrentes)

- `app/actions/post.ts`

- `app/actions/profile.ts`
- `app/actions/workouts.ts`
- `lib/actions/admin.ts`
- `lib/actions/auth.ts`
- `lib/actions/feed.ts`

2.6 Layout raiz e estilos globais

- `app/layout.tsx`
- `app/globals.css`

2.7 Área pública (public)

- `app/(public)/layout.tsx`
- `app/(public)/page.tsx` (home)
- `app/(public)/sobre/page.tsx`
- `app/(public)/equipe/page.tsx`
- `app/(public)/galeria/page.tsx`
- `app/(public)/resultados/page.tsx`
- `app/(public)/contato/page.tsx`

2.8 Área de autenticação (auth)

- `app/(auth)/layout.tsx`
- `app/(auth)/login/page.tsx`
- `app/(auth)/cadastro/page.tsx`
- `app/(auth)/recuperar-senha/page.tsx`

2.9 Área do aluno (dashboard)

- `app/(dashboard)/layout.tsx`
- `app/(dashboard)/dashboard/page.tsx`
- `app/(dashboard)/dashboard/feed/page.tsx`
- `app/(dashboard)/dashboard/treinos/page.tsx`
- `app/(dashboard)/dashboard/perfil/page.tsx`
- `app/(dashboard)/treinos/page.tsx` (rota duplicada)
- `app/(dashboard)/perfil/page.tsx` (rota duplicada)
- `app/(dashboard)/fotos/page.tsx`

2.10 Área administrativa (admin)

- `app/(admin)/layout.tsx`
- `app/(admin)/admin/page.tsx`
- `app/(admin)/admin/treinos/page.tsx`
- `app/(admin)/admin/comunicados/page.tsx`
- `app/(admin)/admin/membros/page.tsx`

2.11 Componentes

- `components/feed/CreatePost.tsx`
- `components/feed/NewPostForm.tsx`
- `components/feed/PostCard.tsx`
- `components/feed/PerfilForm.tsx`

- `components/workouts/CreateWorkoutModal.tsx`
- `components/workouts/WorkoutCard.tsx`
- `components/profile/ProfileForm.tsx`
- `components/admin/AdminForm.tsx`
- `components/layout/Header.tsx`
- `components/layout/Footer.tsx`

2.12 Assets / PWA

- `public/manifest.json`
- `public/logo.png`, `public/robson-portrait.jpg`, `public/robson-running.jpg`, `public/team-group.jpg`
- `public/icons/icon-192.png`, `public/icons/icon-512.png`
- `public/file.svg`, `public/globe.svg`, `public/next.svg`, `public/vercel.svg`, `public/window.svg` (assets padrão do create-next-app, não removidos)
- `app/favicon.ico`

Arquivos binários de imagem foram identificados/listados, mas não abertos byte a byte. `tsconfig.tsbuildinfo` é artefato de build.

3. Comandos Executados e Resultados

3.1 npm install

- **Resultado:** sucesso (exit 0). `added 373 packages, and audited 374 packages in 1m`.
- **Aviso:** `2 moderate severity vulnerabilities`.
- A versão `lucide-react@^1.24.0` **resolve** para `1.24.0` (registry oficial, integridade válida) e **exporta** todos os ícones usados no código (verificado em `node_modules/lucide-react/dist`: `Dumbbell`, `Megaphone`, `Footprints`, `Heart`, `Activity`, `Rss`, etc.). **Não** há falha de instalação de dependência.

3.2 npm audit

```
postcss <8.5.10 – moderate – PostCSS XSS via Unescaped </style> (GHSA-qx2v-qp2m-jg93)
transitivo via next
2 moderate severity vulnerabilities
```

- Correção só via `npm audit fix --force` (breaking). Origem é dependência transitiva do Next.

3.3 npm run lint (eslint)

- **Resultado: FALHA** (exit 1). **24 problemas: 12 erros + 12 warnings**.

Erros (error):

- `app/(auth)/layout.tsx:28` — `2x react/no-unescaped-entities` (aspas " não escapadas).
- `app/(dashboard)/dashboard/page.tsx:76` — `@typescript-eslint/no-explicit-any`.
- `app/(dashboard)/fotos/page.tsx:38` — `2x react/no-unescaped-entities`; `:43` — `no-explicit-any`.
- `app/(dashboard)/treinos/page.tsx:65` — `no-explicit-any`.
- `app/(public)/sobre/page.tsx:106` — `2x react/no-unescaped-entities`.

- components/profile/ProfileForm.tsx:9 e :40 — 2× no-explicit-any .
- components/workouts/CreateWorkoutModal.tsx:24 — no-explicit-any .

Warnings (variáveis/imports não usados):

- app/(admin)/layout.tsx:4 — 'logout' importado e nunca usado.
- app/(dashboard)/dashboard/page.tsx — 'Post' (6), 'user' (47), 'error' (50) não usados.
- app/(dashboard)/dashboard/treinos/page.tsx:4 — 'ChevronRight' não usado.
- app/(public)/page.tsx:12 — 'CheckCircle' não usado.
- app/actions/post.ts:24 — 'data' não usado.
- components/feed/PostCard.tsx:3 — 'MapPin', 'Activity' não usados.
- lib/supabase/server.ts:1 'CookieOptions', :20 'error' não usados.
- middleware.ts:18 — 'options' não usado.

3.4 npx tsc --noEmit (typecheck)

- **Resultado: FALHA** (exit 2). **1 erro de tipo:**

```
app/(dashboard)/dashboard/feed/page.tsx(4,8): error TS2613:
Module '../components/feed/PostCard' has no default export.
Did you mean to use 'import { PostCard } ...' instead?
```

Só aparece **1** erro de tipo porque o cliente Supabase é usado **sem** tipos gerados do banco (Database genérico). Assim, todas as queries com colunas/relacionamentos errados retornam dados fracamente tipados (any / unknown) e **não** geram erro em tempo de compilação — falham somente em runtime.

3.5 npm run build (next build)

- **Resultado: FALHA** (exit 1). O projeto **NÃO builda / NÃO é deployável** no estado atual.
- Sequência: ✓ Compiled successfully in 7.6s → Running TypeScript ... → **Failed to type check.**
- Causa do fracasso: o mesmo TS2613 (PostCard sem default export) em app/(dashboard)/dashboard/feed/page.tsx:4 .
- **Aviso adicional relevante:** The "middleware" file convention is deprecated. Please use "proxy" instead. (Next.js 16). O arquivo middleware.ts usa uma convenção descontinuada.

Conclusão dos comandos: o repositório **não compila até o fim**. O build para no primeiro erro de tipo; portanto podem existir mais erros de tipo ocultos após esse ponto. Lint e typecheck também falham.

4. HANDOVER.md vs. Realidade do Código

O HANDOVER.md descreve um sistema que **diverge significativamente** do código e do schema entregues. Tabela de divergências:

#	Afirmção no HAN-DOVER.md	Realidade no código/schema	Severidade
1	"Next.js 14"	package.json → next 16.2.10 ; README diz Next.js 16	Média (doc desatualizada)
2	role pode ser 'member', 'admin' ou 'treinador'	schema.sql CHECK só permite ('member', 'admin') ; types/index.ts UserRole = 'member' \\ 'admin'	Alta
3	Admin (Samuel) e pai (Robson) precisam de role='admin' ou 'treinador' para ver "Novo Treino"	'treinador' nunca pode existir no banco (viola o CHECK). Todo código que checa role === 'treinador' é morto	Alta
4	Posts têm content e image_url	schema.sql posts tem caption , photo_url , distance_km , duration_minutes , pace , user_id	Crítica
5	Workouts têm assigned_to (treino privado por aluno); "usuários veem só os próprios treinos"	schema.sql workouts não tem assigned_to , só created_by . RLS workouts_select USING (true) → todos os autenticados veem todos os treinos	Crítica
6	Fluxo de recuperação de senha funcional	resetPassword aponta redirectTo para o domínio do Supabase, e rota /recuperar-senha/nova não existe	Alta

O HANDOVER descreve um produto (posts com content/image_url, treinos privados por aluno, papel “treinador”) que **não corresponde** ao banco realmente modelado. Isso indica que a documentação foi escrita sobre uma versão diferente/planejada do sistema, ou que o schema foi alterado depois sem atualizar código e docs.

5. Análise por Área

5.1 Arquitetura geral e DUPLICAÇÃO de código (tema central)

O repositório contém **implementações concorrentes e conflitantes** da mesma funcionalidade. Este é o problema estrutural mais grave.

a) Dois conjuntos de server actions:

- app/actions/ (post.ts , profile.ts , workouts.ts) — **QUEBRADO**, usa colunas que não existem.
- app/actions/post.ts : faz insert com content , image_url , created_by → colunas inexistentes em posts → **falha em runtime**.
- app/actions/workouts.ts : faz insert com assigned_to e checa role === 'treinador' → coluna e papel inexistentes → **falha em runtime**.
- app/actions/profile.ts : atualiza perfil (mais próximo do schema).
- lib/actions/ (feed.ts , admin.ts , auth.ts) — **CORRETO**, casa com o schema.
- lib/actions/feed.ts : createPost / toggleLike / addComment / deletePost com colunas corretas (caption , photo_url , etc.).
- lib/actions/admin.ts : CRUD de workouts/announcements correto; respeita RLS admin.
- lib/actions/auth.ts : logout() , resetPassword() etc.

b) Dois middlewares:

- middleware.ts (raiz) — autocontido, **ativo**. Só verifica se há usuário; **não** força admin em /admin .
- lib/supabase/middleware.ts (updateSession) — **CÓDIGO MORTO**, nunca importado.

c) Rotas duplicadas (mesma feature em 2 lugares):

| Feature | Rota A (linkada na navbar) | Rota B (não linkada) |

|—|—|—|

| Feed | /dashboard (**quebrada**) | /dashboard/feed (correta na query, mas quebra no import do PostCard) |

| Treinos | /treinos (**quebrada**) | /dashboard/treinos (leitura funciona) |

| Perfil | /perfil (usa ProfileForm) | /dashboard/perfil (usa PerfilForm) |

| Fotos | /fotos (**quebrada**) | — |

A sidebar do (dashboard)/layout.tsx aponta para o **conjunto quebrado** (/dashboard , /treinos , /fotos , /perfil). As versões que casam com o schema (/dashboard/feed , /dashboard/treinos , /dashboard/perfil) **não estão** no menu de navegação.

5.2 Autenticação e Autorização

Funciona:

- Setup do Supabase client/server (lib/supabase/client.ts , server.ts) está correto para SSR.
- Cadastro/login via client Supabase nas páginas (auth) .
- Trigger handle_new_user cria o profile automaticamente no signup.
- **Autorização admin server-side:** app/(admin)/layout.tsx busca o profile e checa role === 'admin' , redirecionando caso contrário — **correto e seguro**.

Problemas:

- **Logout quebrado (aluno):** `app/(dashboard)/layout.tsx` envia `<form action="/auth/signout" method="POST">`, mas a rota `/auth/signout` **não existe** (confirmado: só existe `app/auth/callback/route.ts`). O logout do dashboard não funciona. Existe `logout()` em `lib/actions/auth.ts` que resolveria, mas não é usado ali.
- **Middleware não protege /admin :** o `middleware.ts` só verifica se o usuário está logado; a proteção admin depende exclusivamente do layout (que felizmente existe). Não há defesa em profundidade no middleware.
- **Recuperação de senha quebrada:** `lib/actions/auth.ts` → `resetPassword` usa `redirectTo = ${NEXT_PUBLIC_SUPABASE_URL}/auth/callback?next=/recuperar-senha/nova` — aponta para o **domínio do Supabase**, não do app, e a rota `/recuperar-senha/nova` **não existe**. A página `recuperar-senha` também usa client direto em paralelo.
- **Login com elementos mortos:** “Esqueceu a senha?” aponta para `#` (link morto), sem link real para `/recuperar-senha`; checkbox “Lembrar de mim” não está conectado a nada.
- **Duplicação de lógica de auth:** login/cadastro usam o client Supabase diretamente, ignorando as actions de `lib/actions/auth.ts`.
- **Import morto:** `app/(admin)/layout.tsx` importa `logout` e nunca usa (gera warning de lint).

5.3 Banco de Dados — Schema, RLS e Storage (`supabase/schema.sql`)

Tabelas: `profiles`, `posts`, `comments`, `likes`, `workouts`, `announcements`. RLS habilitado em todas.

Colunas reais:

- `profiles`: `id`, `full_name`, `avatar_url`, `bio`, `city`, `role` (CHECK `member / admin`), `goal`, `timestamps`.
- `posts`: `id`, `user_id`, `caption`, `photo_url`, `distance_km`, `duration_minutes`, `pace`, `created_at`.
- `workouts`: `id`, `created_by`, `title`, `description`, `date`, ... **sem** `assigned_to`.
- `comments`, `likes`, `announcements`.

Funções/Triggers:

- `is_admin()` — SECURITY DEFINER, usada nas policies.
- Trigger `updated_at` e `handle_new_user` (cria profile no signup).

Storage: buckets `avatars` e `post-images` com policies de upload/leitura.

Problemas de modelagem/segurança:

- **profiles sem policy de DELETE:** `deleteMember` em `lib/actions/admin.ts` tenta deletar profile, mas sem policy de DELETE a RLS **bloqueia silenciosamente** (0 linhas afetadas) → funcionalidade de remover membro **não funciona** e não avisa erro.
- **profiles_update_admin sem WITH CHECK:** policy de UPDATE para admin define só `USING`, sem `WITH CHECK`, permitindo potencialmente gravar valores fora do escopo pretendido.
- **RLS de workouts contradiz o HANDOVER:** `workouts_select USING (true)` significa que **todo** usuário autenticado vê **todos** os treinos. A promessa de “treino privado por aluno” é falsa no schema atual.
- **Sem tipos gerados (Database):** o client Supabase não usa tipos do banco, então erros de coluna não são detectados em compile-time (ver §3.4).

5.4 Feed / Posts / Curtidas / Comentários

Estado: QUEBRADO em ambas as rotas.

- `/dashboard` (feed principal linkado): `app/(dashboard)/dashboard/page.tsx`

- Query usa `profiles!posts_created_by_fkey` — **FK inexistente** (o vínculo real é `posts.user_id → auth.users`).
- Lê `post.content` e `post.image_url` — **colunas inexistentes** (são `caption / photo_url`).
- Curtidas e comentários **hardcoded em 0** com comentário “To implement”.
- `/dashboard/feed (feed alternativo)`: `app/(dashboard)/dashboard/feed/page.tsx`
- A query casa com o schema (usa `lib/actions/feed.ts`), **porém** importa `PostCard` como **default** (`import PostCard from ...`), e o componente só tem **named export** → `TS2613` → **falha o build** e daria “Element type is invalid” em runtime.
- `components/feed/PostCard.tsx` : só possui `export function PostCard(...)` com props **estáticas** (`authorName`, `authorInitials`, `timeAgo`, `content`, `imageUrl`, `distance`, `pace`, `likes`, `comments`). Os botões de curtir/comentar **não têm onClick** — não estão ligados a nenhuma ação.
- **Ações mortas**: `lib/actions/feed.ts` (`toggleLike`, `addComment`, `deletePost`) existem e são corretas, mas **nenhum componente as chama** → código morto.
- **Componentes de criação de post concorrentes**: `components/feed/CreatePost.tsx` e `components/feed/NewPostForm.tsx` coexistem (duplicação); ligados a actions diferentes.

Resultado líquido: curtir, comentar e (dependendo da rota) criar/visualizar posts **não funcionam**. O feed é, na prática, estático/quebrado.

5.5 Treinos

- `/treinos (linkado na navbar)`: `app/(dashboard)/treinos/page.tsx`
- Query usa `profiles!workouts_created_by_fkey` — **FK inexistente** (não há relação `workouts → profiles`).
- Textos afirmam “treinos enviados exclusivamente para você pelo seu treinador” e “Seu treinador ainda não designou treinos” → **enganoso**: não há `assigned_to`; RLS mostra todos os treinos a todos.
- Checa `role === 'treinador'` (papel impossível).
- `/dashboard/treinos` : leitura de treinos funciona (query mais alinhada), sem privacidade por aluno.
- **Criação**: `app/actions/workouts.ts` insere `assigned_to` + checa `treinador` → **falha**. `lib/actions/admin.ts` cria workout corretamente (usado no painel admin).
- `components/workouts/CreateWorkoutModal.tsx` : texto “Apenas o treinador e este atleta poderão ver este treino” → **falso** (sem privacidade no schema); usa `any` (erro de lint).
- `components/workouts/WorkoutCard.tsx` : exibição OK.

5.6 Comunicados (announcements)

- **Admin**: `app/(admin)/admin/comunicados/page.tsx` + `lib/actions/admin.ts` → CRUD **correto** e alinhado ao schema; RLS exige admin. Funciona.
- **Consumo pelo aluno**: não há uma tela de aluno dedicada e funcional consumindo announcements no fluxo de navegação principal (comunicados aparecem, na melhor das hipóteses, de forma limitada). Verificar integração com o dashboard do aluno na fase de reconstrução.

5.7 Painel Admin / Membros

Funciona (bem modelado):

- `app/(admin)/layout.tsx` : autorização admin server-side correta.
- `app/(admin)/admin/page.tsx` : dashboard admin com contadores/links.
- `admin/treinos` e `admin/comunicados` : CRUD via `lib/actions/admin.ts`, correto.

Problemas:

- `admin/membros` (`app/(admin)/admin/membros/page.tsx`): promover/rebaixar role pode funcionar (UPDATE tem policy admin), **mas remover membro NÃO funciona** — `deleteMember` esbarra na ausência de policy DELETE em `profiles` (falha silenciosa).
- Import morto de `logout` no layout admin.

5.8 Perfis / Uploads

Funciona:

- Atualização de perfil casa com o schema em ambos os formulários.
- Upload de avatar para o bucket `avatars` funciona (`PerfilForm.tsx` e `ProfileForm.tsx`).

Problemas:

- **Dois formulários concorrentes:** `components/feed/PerfilForm.tsx` (usado em `/dashboard/perfil`) vs `components/profile/ProfileForm.tsx` (usado em `/perfil`). Duplicação; `ProfileForm.tsx` usa `any` (2 erros de lint).
- **/fotos quebrada:** `app/(dashboard)/fotos/page.tsx` consulta a coluna `image_url` (inexistente; é `photo_url`) → falha; ainda tem 2 erros de aspas não escapadas + `any`.

5.9 Navegação e Rotas

- Sidebar do aluno aponta para o conjunto **quebrado** de rotas (§5.1c).
- Header/Footer públicos (`components/layout/`) usam âncoras hash (`#sobre` , `#equipe` , etc.) que **só existem na home**; em outras páginas esses links não navegam para lugar nenhum útil.
- Telefones placeholder inconsistentes: a maioria usa `5519900000000` ; o WhatsApp desktop do Header usa `5519999999999` .

5.10 CSS / Estilização / Design System (quebra visual ampla)

Configuração conflitante entre Tailwind v3 e v4:

- `postcss.config.mjs` usa `@tailwindcss/postcss` (v4). `globals.css` usa `@import 'tailwindcss'` + bloco `@theme inline` definindo tokens `--color-btr-*`.
- `tailwind.config.ts` está em **estilo v3** (define `brand.*`, fonte `Outfit`, `shadow-card*`), mas **não é carregado** — não há diretiva `@config` no CSS, então em Tailwind v4 ele é **ignorado/morto**.

Classes/variáveis CSS referenciadas mas NÃO definidas (resultam em estilo quebrado):

- Classes utilitárias custom: `card`, `btn-primary`, `btn-secondary`, `btn-outline`, `badge`, `badge-red`, `badge-green`, `badge-orange`, `badge-gray`, `section-title`, `section-subtitle`, `input-base`, `divider-red`, `divider-green`, `divider-orange`, `shadow-card-lg`.
- Variáveis CSS: `var(--color-red)`, `var(--color-green)`, `var(--color-orange)`, `var(--color-bg)` — **nenhuma** definida em `globals.css`.

Páginas/componentes afetados pela quebra de estilo: `sobre`, `equipe`, `galeria`, `resultados`, `contato`, `layout` e página `recuperar-senha` de `(auth)`, `dashboard/treinos`, `dashboard/perfil`, `dashboard/feed`, **todas** as páginas de `(admin)`, e `NewPostForm`, `PerfilForm`, `AdminForm`.

Funciona visualmente: a home (`app/(public)/page.tsx`) usa hex hardcoded (`[#DC2626]`), então renderiza mesmo sem os tokens.

Fontes inconsistentes: `app/layout.tsx` carrega `Barlow Condensed` + `Inter`; `tailwind.config.ts` referencia `Outfit`; `globals.css` referencia `Barlow Condensed`. Sem alinhamento.

5.11 Páginas Públicas (conteúdo estático / mock)

Todo o conteúdo de marketing é **estático/hardcoded** (esperado para landing, mas registrado):

- Home: estatísticas fictícias (“500+ atletas”, “98%”), depoimentos, textos.
- Galeria: fotos de `Unsplash` (URLs externas) hardcoded.
- Equipe/Sobre/Resultados/Contato: textos e números fixos, formulário de contato sem backend real (verificar submissão).

5.12 Tratamento de Erros

- Diversas queries ignoram o objeto `error` do Supabase (warnings de `error` não usado em `dashboard/page.tsx`, `lib/supabase/server.ts`).
- Curtidas/comentários “To implement” com valores fixos em vez de tratamento real.
- Falhas silenciosas: `deleteMember` (RLS bloqueia sem feedback).
- Ausência de páginas `error.tsx` / `not-found.tsx` / `loading.tsx` no App Router (nenhuma encontrada) → sem tratamento de erro/boundary ou estados de carregamento padronizados.

5.13 Validação de Formulários

- Validações são majoritariamente client-side básicas (HTML `required`), sem uma camada de validação de schema (ex.: `zod`) nem no client nem nas server actions.
- Formulários de post/treino/perfil não validam limites (tamanho de imagem, formato, comprimento de campos) de forma consistente.

5.14 Tipagem TypeScript

- **1 erro de tipo bloqueante** (`PostCard` default export) que **quebra o build**.
- Uso disseminado de `any` (erros de lint em `dashboard/page.tsx`, `fotos/page.tsx`, `treinos/page.tsx`, `ProfileForm.tsx`, `CreateWorkoutModal.tsx`).
- Cliente Supabase **sem tipos** `Database` → erros de coluna/relacionamento não são pegos em compile-time (mascaram bugs de runtime).
- `types/index.ts` define `UserRole = 'member' | 'admin'` (sem `treinador`), divergindo do HAN-DOVER e do código que usa `'treinador'`.

5.15 Dependências (instaladas vs usadas)

- Dependências diretas mínimas: `@supabase/ssr`, `@supabase/supabase-js`, `lucide-react`, `next`, `react`, `react-dom`. Todas **usadas**.
- `devDeps`: `@tailwindcss/postcss`, `@types/*`, `eslint`, `eslint-config-next`, `tailwindcss`, `typescript`. Coerentes.
- `lib/utils.ts` implementa `cn` manualmente (não usa `clsx` / `tailwind-merge`); nenhuma dessas libs está nas deps e nenhuma é referenciada → OK, sem import quebrado.
- `lucide-react@^1.24.0`: resolve e funciona (§3.1), embora a especificação de versão seja incomum — vale fixar/validar na reconstrução.
- 2 vulnerabilidades moderadas transitivas (`postcss` via `next`).

5.16 Configurações e PWA

- `next.config.ts`: mínimo.
- `middleware.ts`: usa convenção **descontinuada** no Next 16 (aviso para migrar para `proxy`).
- `public/manifest.json`: PWA válido, com atalhos para `/dashboard` e `/dashboard/feed`.
- **Sem** `.env.example`: o README instrui criar `.env.local` manualmente com as chaves do Supabase; a ausência de um exemplo versionado dificulta o onboarding.
- Assets padrão do `create-next-app` (`next.svg`, `vercel.svg`, etc.) não foram removidos.

6. Inventário de Bugs por Severidade

● Crítico (impede build/uso central)

1. **Build falha** — `PostCard` sem default export, importado como default em `dashboard/feed/page.tsx` (TS2613). O projeto não builda/deploya.
2. **Feed principal quebrado** (`/dashboard`) — FK inexistente `posts_created_by_fkey` + colunas `content` / `image_url` inexistentes; curtidas/comentários hardcoded.
3. **Criação de post quebrada** (`app/actions/post.ts`) — insere colunas inexistentes.
4. **Criação de treino quebrada** (`app/actions/workouts.ts`) — insere `assigned_to` inexistente e checka papel `treinador` impossível.
5. **/fotos quebrada** — consulta coluna `image_url` inexistente.
6. **Curtir/comentar não funcionam** — botões sem handler; actions corretas existem mas estão mortas.

● Alto

1. **Logout do aluno quebrado** — POST para `/auth/signout` inexistente.
2. **Recuperação de senha quebrada** — `redirectTo` para domínio errado + rota `/recuperar-senha/nova` inexistente.
3. **Remover membro não funciona** — falta policy DELETE em `profiles` (falha silenciosa).
4. **Papel treinador impossível** — CHECK do schema só permite `member` / `admin` ; docs e código dependem de `treinador` .
5. **Privacidade de treino inexistente** — RLS `USING (true)` expõe todos os treinos a todos (contradiz `HANDOVER` e a UI).
6. **Sidebar aponta para rotas quebradas** — navegação principal do aluno leva ao conjunto defeituoso.

● Médio

1. **Quebra de estilo ampla** — dezenas de classes/variáveis CSS indefinidas (Tailwind config v3 morto sob v4).
2. **Duplicação estrutural** — dois conjuntos de actions, dois middlewares, rotas/formulários duplicados.
3. **Lint falha** — 12 erros (`no-explicit-any` , `no-unescaped-entities`).
4. **Sem tipos Database do Supabase** — erros de coluna passam despercebidos em compile-time.
5. `profiles_update_admin` sem `WITH CHECK` .
6. **Middleware não reforça admin** (defesa só no layout) + convenção `middleware` descontinuada (Next 16).
7. **Sem** `error.tsx` / `not-found.tsx` / `loading.tsx` .

● Baixo

1. **Documentação divergente** (`HANDOVER`: Next 14; versões/colunas erradas).
2. **Telefones placeholder inconsistentes** (`...9000000000` vs `...9999999999`).
3. **Links hash mortos** no Header/Footer fora da home; “Esqueceu a senha?”/“Lembrar de mim” não conectados.
4. **12 warnings de lint** (imports/variáveis não usados).
5. **Sem** `.env.example` ; assets padrão do create-next-app não removidos.
6. **Inconsistência de fontes** (Barlow/Inter vs Outfit).

7. Resumo: Funciona vs. Mock/Estático vs. Quebrado

✓ Funciona (conectado ao Supabase, alinhado ao schema)

- Setup Supabase SSR (client/server).
- Cadastro e login (via client Supabase).
- Trigger `handle_new_user` (cria profile no signup).
- **Autorização admin** no `(admin)/layout.tsx` (server-side, segura).
- **CRUD admin** de treinos e comunicados (`lib/actions/admin.ts`).
- Atualização de perfil e **upload de avatar** (bucket `avatars`).
- Leitura de treinos em `/dashboard/treinos` (sem privacidade por aluno).
- PWA/manifest válido.

🟡 Mock / Estático (esperado, mas registrado)

- Todas as páginas públicas (home, sobre, equipe, galeria, resultados, contato): números, depoimentos e fotos (Unsplash) hardcoded.
- Formulário de contato sem backend confirmado.

🔴 Quebrado / Não funciona

- **Build/deploy** (falha de tipo).
- Feed principal `/dashboard` (colunas/FK erradas).
- `/dashboard/feed` (import quebrado do PostCard).
- Criar post (`app/actions/post.ts`).
- Curtir e comentar (sem handlers; actions mortas).
- Criar treino via `app/actions/workouts.ts` (coluna/papel inexistentes).
- `/fotos` (coluna inexistente).
- Logout do aluno (rota inexistente).
- Recuperação de senha (redirect/rota erradas).
- Remover membro (sem policy DELETE).
- Estilização de grande parte das páginas internas e públicas secundárias (classes CSS indefinidas).

🤖 Código morto / duplicado

- `lib/supabase/middleware.ts` (`updateSession`) — nunca importado.
- `lib/actions/feed.ts` (`toggleLike` / `addComment` / `deletePost`) — corretas, mas não chamadas.
- `lib/actions/auth.ts` (`logout`) — não usada onde precisaria.
- Rotas duplicadas `/dashboard` vs `/dashboard/feed` , `/treinos` vs `/dashboard/treinos` , `/perfil` vs `/dashboard/perfil` .
- `tailwind.config.ts` — não carregado sob Tailwind v4.
- Componentes concorrentes: `CreatePost` vs `NewPostForm` ; `ProfileForm` vs `PerfilForm` .
- Imports não usados (vários; ver §3.3).

8. Observações Finais para a Fase de Reconstrução

- O código entregue parece resultado de **duas gerações sobrepostas** (um conjunto “planejado” descrito no HANDOVER com `content / image_url / assigned_to / treinador`, e um conjunto “real” alinhado ao schema em `lib/actions/`), sem que a fusão tenha sido concluída. A navegação aponta para a metade errada.
 - **Fonte da verdade recomendada:** o `supabase/schema.sql` (colunas reais) e as actions de `lib/actions/`. As actions de `app/actions/` e as páginas `/dashboard`, `/treinos`, `/fotos`, `/perfil` estão desalinhadas.
 - Antes de qualquer reconstrução: definir claramente o modelo de papéis (incluir ou não `treinador`), o modelo de privacidade de treinos (`assigned_to` + RLS por aluno, se for requisito), e consolidar o design system em Tailwind v4 (definir todos os tokens/classes em `globals.css` ou migrar o `tailwind.config.ts` com `@config`).
-

Fim do AUDIT_RAW_FINDINGS.md — nenhum arquivo-fonte foi modificado durante esta auditoria.